

Introduction

- Code optimization (or code improvement)
 - Eliminating **unnecessary** instructions in object code
 - Replacing one sequence of instructions by a **faster** sequence of instructions that does the same thing
- Global optimizations
 - Most global optimizations are based on **data-flow analyses**
 - Data-flow analyses: algorithms to gather info about a program
- A compiler knows only how to apply relatively **low-level semantic transformations** to optimize code

代码优化（或代码改进）

消除目标代码中不必要的指令

用做同样事情的更快指令序列替代一个指令序列

全局优化

大多数全局优化都基于数据流分析

数据流分析：收集程序信息的算法

编译器只知道如何应用相对低级的语义转换来优化代码

Two Major Causes of Redundancy

1. Redundant operations at the **source level** (e.g., introduced by programmers for convenience)
2. **Side effect** of writing programs in a high-level language
 - In most languages (other than C/C++, which allows pointer arithmetic), programmers can only refer to **elements of an array** or **fields in a structure** through accesses like $A[i][j]$ or $X \rightarrow f1$
 - During compilation, each high-level data-structure access expands into **a sequence of low-level arithmetic operations**
 - Access to the same data structure often create many common low-level operations

典型的程序中存在很多冗余操作。

1.有时，在源代码中会用到冗余。例如，程序员可能会发现重新计算某些结果更加直接和方便，而让编译器去发现实际上只需要进行一次这样的计算。

2.但更多的时候，冗余是使用高级语言编写程序的副产品。

- 在大多数语言中（不包含 C 或 C++，它们允许对指针进行算术运算），程序员别无选择，只能通过 $A[i][j]$ 或 $X \rightarrow f1$ 之类的访问来引用数组的元素或结构中的字段。

- 当程序被编译时，每一个这样的高级数据结构访问都会被扩展为多个低层次的算术运算，例如计算一个矩阵 A 的第 (i; j) 个元素的位置的运算。

- 对于同一个数据结构的访问通常共享许多公共的低级运算。程序员不知道这些低级运算，也无法自行消除这些冗余。事实上，从软件工程的角度来看，程序员最好只通过数据元素的高级名字来访问它们。这样程序更容易编写，更重要的是，更容易理解和演化。通过让编译器消除冗余，我们可以两全其美：程序既高效又易于维护。

A Running Example: Quicksort

```
void quicksort(int m, int n)
/* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x; /* swap a[i], a[j] */
    }
    x = a[i]; a[i] = a[n]; a[n] = x; /* swap a[i], a[n] */
    /* fragment ends here */
    quicksort(m, j); quicksort(i+1, n);
}
```

(1)	i = m-1	(16)	t7 = 4*i
(2)	j = n	(17)	t8 = 4*j
(3)	t1 = 4*n	(18)	t9 = a[t8]
(4)	v = a[t1]	(19)	a[t7] = t9
(5)	i = i+1	(20)	t10 = 4*j
(6)	t2 = 4*i	(21)	a[t10] = x
(7)	t3 = a[t2]	(22)	goto (5)
(8)	if t3<v goto (5)	(23)	t11 = 4*i
(9)	j = j-1	(24)	x = a[t11]
(10)	t4 = 4*j	(25)	t12 = 4*i
(11)	t5 = a[t4]	(26)	t13 = 4*n
(12)	if t5>v goto (9)	(27)	t14 = a[t13]
(13)	if i>=j goto (23)	(28)	a[t12] = t14
(14)	t6 = 4*i	(29)	t15 = 4*n
(15)	x = a[t6]	(30)	a[t15] = x

Three-address code

接下来，我们将使用名为快速排序的排序程序片段来说明几个重要的可以改进代码的转换。

图 9.1 中的 C 程序源自 Sedgwick，它讨论了如何对这样一个程序进行手工优化。我们不会在这里讨论这个程序在算法方面的所有微妙的细节，例如，a[0] 必然存放着已排序的元素的最小者，而 a[max]则存放最大的元素。

在我们可以优化掉地址计算中的冗余之前，必须首先将程序中的地址运算分解为低级算术运算，这样才能暴露处冗余之处。在本章的其余部分中，我们假设中间表示形式由三地址语句组成，其中所有中间表达式的结果都由临时变量来保存。图 9.1 中标记的程序片段的中间代码如图 9.2 所示。

在这个例子中，我们假设整数占用四个字节。

Array Accesses

```
(1)  i = m-1
(2)  j = n
(3)  t1 = 4*n
(4)  v = a[t1]
(5)  i = i+1
(6)  t2 = 4*i
(7)  t3 = a[t2]
(8)  if t3 < v goto (5)
(9)  j = j-1
(10) t4 = 4*j
(11) t5 = a[t4]
(12) if t5 > v goto (9)
(13) if i >= j goto (23)
(14) t6 = 4*i
(15) x = a[t6]
(16) t7 = 4*i
(17) t8 = 4*j
(18) t9 = a[t8]
(19) a[t7] = t9
(20) t10 = 4*j
(21) a[t10] = x
(22) goto (5)
(23) t11 = 4*i
(24) x = a[t11]
(25) t12 = 4*i
(26) t13 = 4*n
(27) t14 = a[t13]
(28) a[t12] = t14
(29) t15 = 4*n
(30) a[t15] = x
```

$x = a[i] \rightarrow$ (14), (15)

$a[j] = x \rightarrow$ (20), (21)

Each array access translates into a multiplication and an array-subscripting operation

赋值运算 $x = a[i]$ 按照第 6.4.4 节翻译为两个三地址语句 (14)、(15)

类似的, $a[j] = x$ 变成了 (20)、(21)

请注意, 原始程序中的每个数组访问都会转换为一对语句, 包括一个乘法和一个数组下标运算。结果, 这个短短的程序片段被翻译成一个相当长的三地址运算序列。

Flow Graph

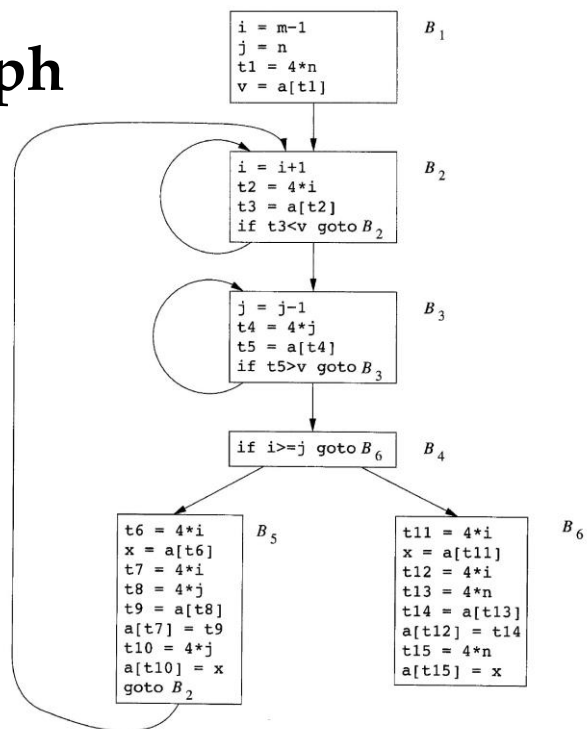


图 9.3 是图 9.2 中程序的流图。

B1 块是其入口结点。

在 8.4 节介绍过，图 9.2 中所有条件和无条件跳转语句的目标在图 9.3 中都被替换为以它们的为目标语句为首语句的基本块。

在图 9.3 中有 3 个循环。块 B2 和 B3 本身就是循环。块 B2、B3、B4 和 B5 一起形成一个循环，其中 B2 是唯一的入口点。

Semantic-Preserving Transformations

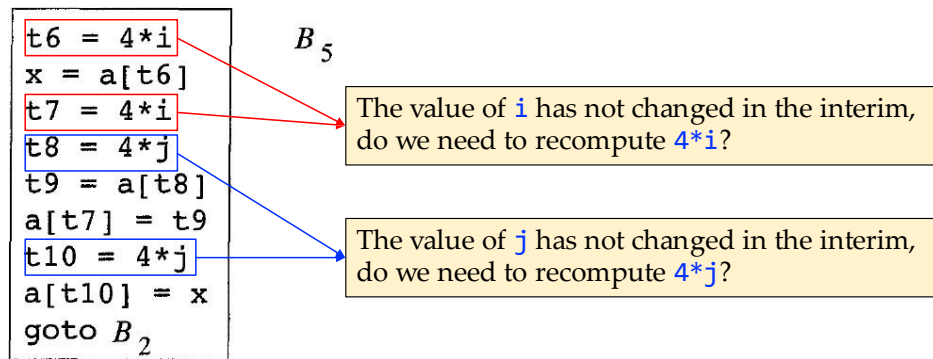
- Common-subexpression elimination
- Copy propagation
- Dead-code elimination
- Code motion

9.1.3 保持语义不变的转换

编译器可以使用多种方法改进一个程序，但不改变程序所计算的函数。

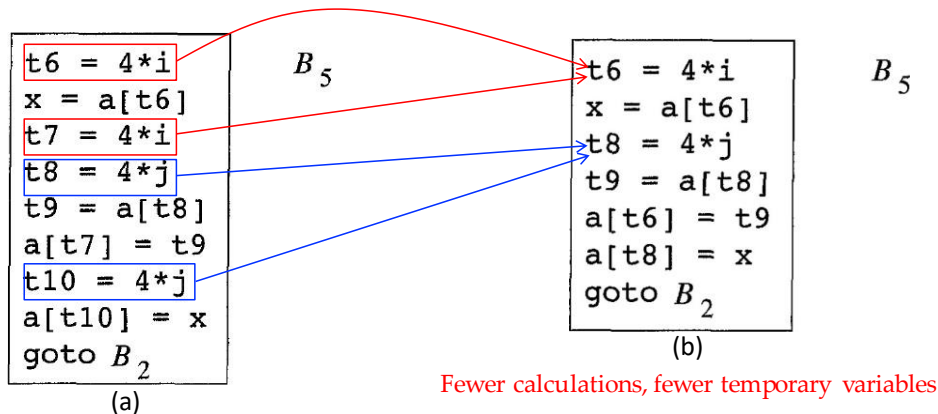
公共子表达式消除、复制传播、死代码消除和常量折叠都是这样的函数不变（或者说语义不变）转换的常见例子。我们将逐一介绍这些方法。

Local Common-Subexpression Elimination



局部公共子表达删除：通常，程序会包含对相同值的多次计算，例如计算数组中的偏移量。例如，图 9.4(a) 中所示的块 B_5 中对 $4*i$ 和 $4*j$ 进行了重复计算，尽管这些计算全都不是程序员显式要求的。

Local Common-Subexpression Elimination

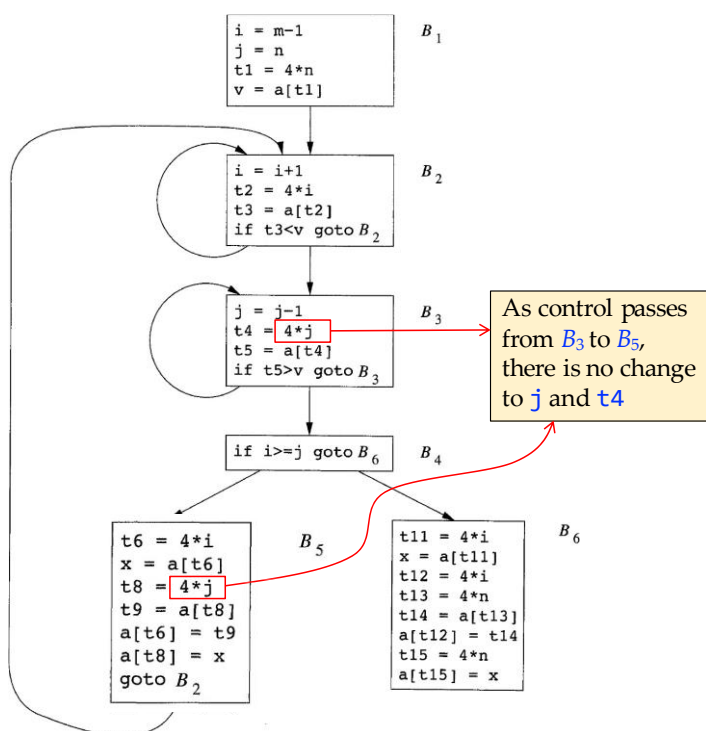


Such duplicate calculations cannot be avoided by programmers because they lie below the level of detail accessible within the source language

正如第 9.1.2 节中提到的，某些这样的重复计算，不可能由程序员来避免，因为这些计算过程处于可在源语言中处理的细节的更下层。如果表达式 E 是先前计算过的，并且 E 中变量的值自上次计算以来没有改变，则表达式 E 的该次出现就称为公共子表达式。如果将 E 的上一次计算结果赋予变量 x ，且 x 的值在此期间没有改变，我们就可以使用前面计算得到的值，从而避免重新计算 E 。

例 9.1：图 9.4(a) 中对 $t7$ 和 $t10$ 的赋值，分别计算了公共子表达式 $4*i$ 和 $4*j$ 。这些步骤已经在图 9.4(b) 中被消除了。消除后的代码使用 $t6$ 代替 $t7$ ，使用 $t8$ 代替 $t10$ 。

Global Common Subexpression Elimination



全局公共子表达式删除：我们首先讨论 B_5 的转换，然后再讨论一些涉及数组相关的精妙之处。

消除局部公共子表达式后， B_5 仍对 $4*i$ 和 $4*j$ 进行求值，如图 9.4 (b) 所示。两者都是公共子表达式；更明确的讲，使用块 B_3 中计算得到的 t_4 的值， B_5 中的三个语句：

$t_8 = 4*j$

$t_9 = a[t_8]$

$A[t_8] = x$

可以替换为

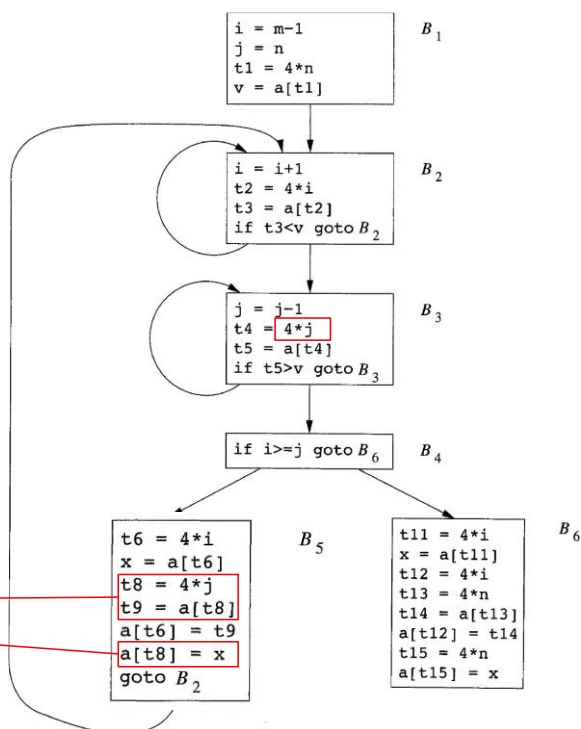
$t_9 = a[t_4]$

$a[t_4] = x$

在图 9.5 中，观察到当控制从 B_3 中计算 $4*j$ 的点传递到 B_5 中时， j 没有变化， t_4 也没有变化，因此如果需要 $4*j$ 时，可以使用 t_4 来替代。

Global Common Subexpression Elimination

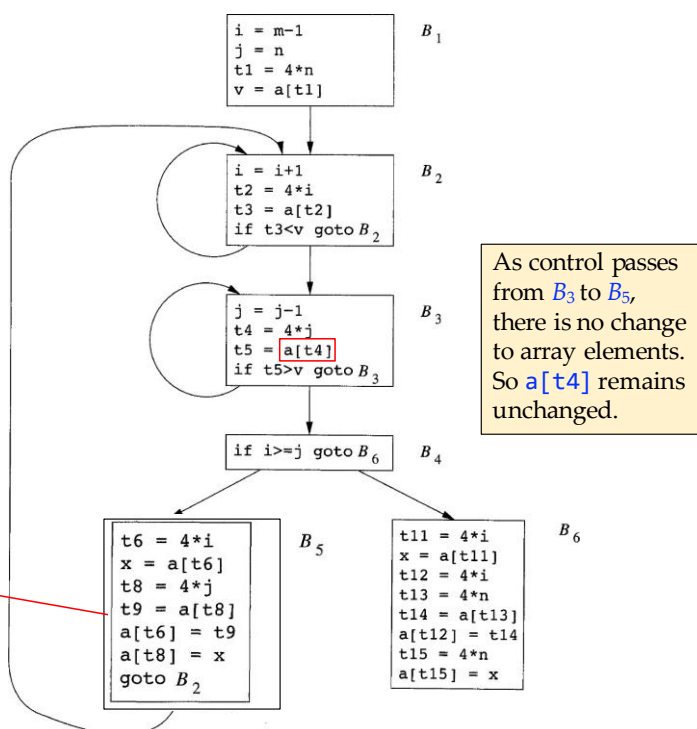
Optimized:
 $t9 = a[t4]$
 $a[t4] = x$



在用 $t4$ 替换 $t8$ 后, $B5$ 中出现了另一个公共的子表达式 $a[t4]$ 。

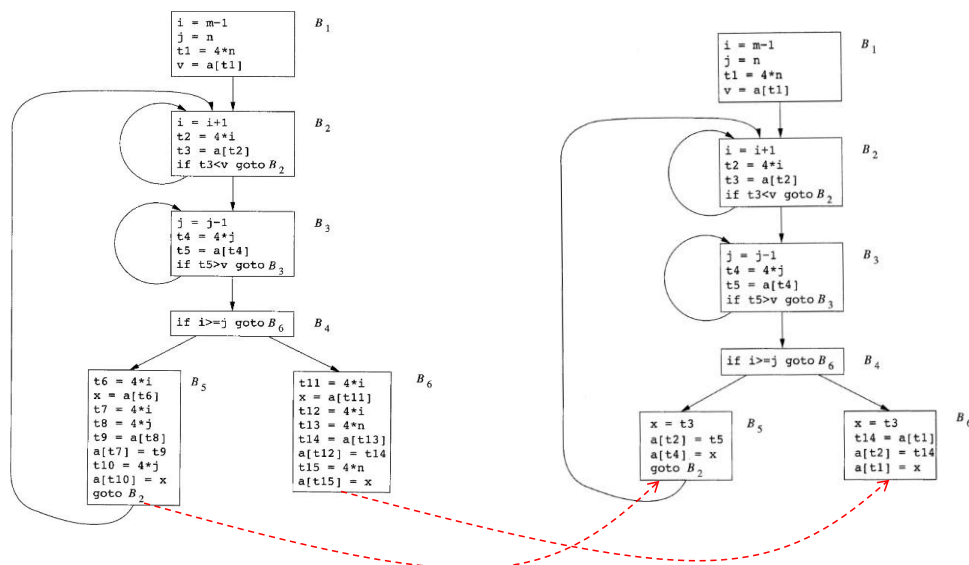
Global Common Subexpression Elimination

Optimized:
 $a[t6] = t5$



新的子表达式 $a[t4]$ 对应于源代码层次上的值 $a[j]$ 。当控制离开 $B3$ 然后进入 $B5$ 时，不仅仅 j 保留了它的值，而且 $a[j]$ 也保留了原来的值。这个值在计算出来之后保存到临时变量 $t5$ 中。因为在此期间没有对数组 a 的元素进行赋值。因此， $a[j]$ 的值不变。 $B5$ 中的语句 $a[t6] = t9$ 可以被替换为 $a[t6] = t5$

After Optimizations



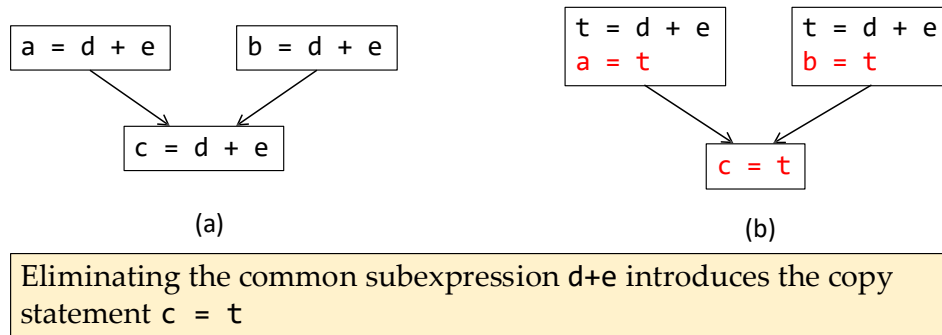
类似地，在图 9.4(b) 的块 $B5$ 中赋给 x 的值 ($x=a[t6]$) 和 $B2$ 中赋给 $t3$ 的值 ($t3=a[t2]$) 相同。图 9.5 中的块 $B5$ 是从图 9.4(b) 中的 $B5$ 中消除了与源代码级表达式 $a[i]$ 和 $a[j]$ 的值对应的公共子表达式之后的结果。图 9.5 中的 $B6$ 也进行了一系列类似的变换。

图 9.5 的块 $B1$ 和 $B6$ 中的表达式 $a[t1]$ 不被视为公共子表达式，尽管 $t1$ 可以在两个地方使用。在控制离开 $B1$ 后、到达 $B6$ 之前，它还可能经过 $B5$ ，而 $B5$ 中有对 a 的赋值。因此， $a[t1]$ 在到达 $B6$ 时可能和它离开 $B1$ 时的值不同。将 $a[t1]$ 视为公共子表达式是不安全的。

图 9.5 中的块 $B5$ 可以通过使用两个新的变换消除 x ，从而得到进一步改进。其中一个转化考虑形如 $u = v$ 的赋值表达式，这种表达式被称为复制语句，简称复制。如果我们在例 9.2 中更详细地讨论，很快就会发现一些复制语句。因为消除公共子表达式的常用算法会引入复制语句，其他几种优化算法也会引入这样的语句。

Copy Propagation

- Optimization algorithms often introduce *copy statements*
 - Form: $u = v$



去掉公共子表达式 $d+e$ 后，就引入了复制语句 $c = t$ (复制语句，即形如 $x = y$ 的赋值语句)

常用的公共子表达式消除算法和其它一些优化算法会引入一些复制语句

例 9.3: 为了消除图 9.6(a) 中的公共子表达式语句 $c = d+e$ ，我们必须使用一个新变量 t 来保存 $d+e$ 的值。在图 9.6(b) 中，变量 t 的值而不是表达式 $d+e$ 的值被赋给 c 。由于控制可能经过对 a 的赋值到达 $c = d+e$ 处，也可能经过对 b 的赋值到达 $c = d+e$ 处，因此用 $c = a$ 或 $c = b$ 替换 $c = d+e$ 是不正确的。

Copy Propagation

- Copy propagation is to **use v for u**, wherever possible after the copy statement



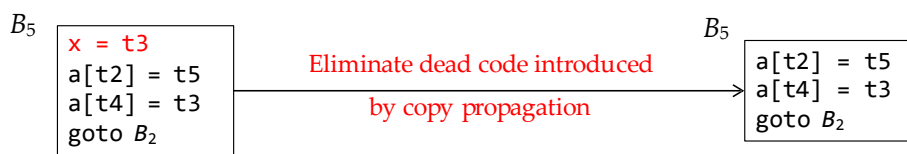
This change may not appear to be an improvement, but it gives us the opportunity to eliminate the assignment to x

复制传播变换背后的思想是在复制语句 $u = v$ 之后尽可能使用 v 代替 u 。

例如，图 9.5 的块 B_5 中的赋值语句 $x = t3$ 是一个复制语句。应用于 B_5 的复制传播产生图 9.7 中的代码。这一变化可能看起来不是一种改进，但是，正如我们将在第 9.1.6 节中看到的，它为我们提供了消除对 x 赋值的语句的机会。

Dead-Code Elimination

- A variable is *live* at a program point if its value can be used subsequently; otherwise, it is *dead* at that point
- *Dead code*: Statements computing values that are never used
- Dead code appears often because of optimizations



如果一个变量在某一程序点上的值可能会在以后使用，那么我们就说这个变量在该点是活跃的。否则，它在该点就是死的。与此相关的一个想法就是死（或无用）代码。

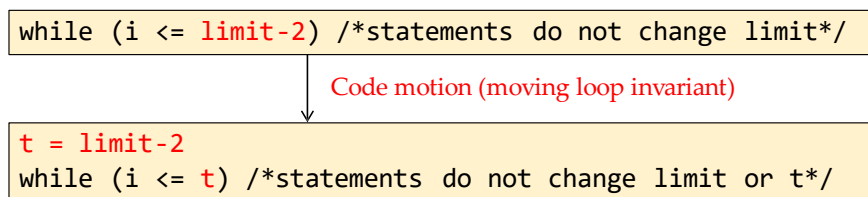
所谓死代码就是其计算结果永远都不会被使用的语句。

虽然程序员不太可能故意引入死代码，死代码多半是因为前面执行过的某些转换而造成的。

复制传播的优点之一是它经常将复制语句变成死代码。例如，先进行复制传播再进行死代码消除，就可以去掉图 9.7 (左图) 中的代码对 `x` 的赋值，并将转换为 (右图)

Code Motion

- Loops are a very important place for optimizations
- **Loop invariant**: an expression that yields the same result regardless of the number of times a loop is executed
- Moving loop invariants and evaluate them before loops can decrease the running time of a program



循环是优化的一个非常重要的地方，特别是程序往往花费大量时间的内部循环。如果我们减少内部循环中的指令数量，即使我们增加该循环外部的代码量，程序的运行时间也可能会得到改善。

减少循环内部代码数量一个重要改动是代码移动。此转换处理的是那些不管循环执行多少次都得到相同结果的表达式（循环不变计算），在进入循环之前就对它们求值。

请注意，概念“在循环之前”的说法假定存在一个循环入口。所谓循环入口就是一个基本块（请参见第 8.4.5 节）所有循环外部到循环的跳转指令都以它为目标。

示例 9.5：在下面的 while 语句中，对 $\text{limit} - 2$ 的求值是一个循环不变计算：

现在， $\text{limit} - 2$ 的计算只在进入循环之前被执行一次。之前，如果我们重复循环体 n 次，将会对 $\text{limit} - 2$ 计算 $n + 1$ 次。

循环是进行优化的重要场所

循环不变式：无论循环执行多少次，结果都相同的表达式

移动循环不变式并在循环前对其进行评估可减少程序的运行时间

What Is Data-Flow Analysis?

- A body of techniques that derive information about the flow of data along program execution paths
 - **Example 1:** Whether two textually identical expressions evaluate to the same value along any execution path? (Identify common subexpressions)
 - **Example 2:** Whether the result of an assignment is used on subsequent execution paths? (Eliminate dead code)
- Data-flow analysis is the foundation of many optimizations

“数据流分析”是指一组用来获取有关数据如何沿着程序执行路径流动的相关信息的技术。

例如，实现全局公共子表达式消除的一种方法要求我们确定在程序的任何可能的执行路径上，两个字面相同的表达式是否会给出相同的值。(找出共同的子表达式)

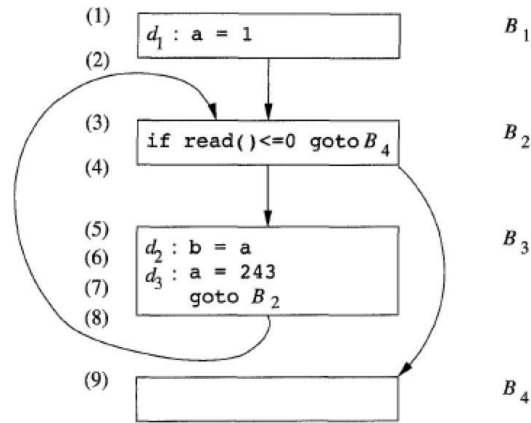
另一个例子是，如果某一个赋值语句的结果在任何后续执行路径中都没有被使用，那么我们可以将赋值作为死代码消除。(消除死代码)

这些和许多其他重要问题都可以通过数据流分析来回答。数据流分析是许多优化的基础。

The Data-Flow Abstraction

- **Program points**: points before and after each statement

- Within one block, the program point after a statement is the same as the program point before the next statement (e.g., 6)
- If there is an edge from block B to block C , then the program point after the last statement of B **may be followed immediately by** the program point before the first statement of C (e.g., 8 and 3)



数据流抽象

程序的执行可以看作是对程序状态的一系列转换。程序状态由程序中的所有变量的值组成，同时包括运行时刻的栈顶之下各个帧的相关值。一个中间代码语句的每次执行都会把一个输入状态转换成一个新的输出状态。这个输出状态和处于该语句之前的程序点相关联，而输出状态和该语句之后的程序点相关联。

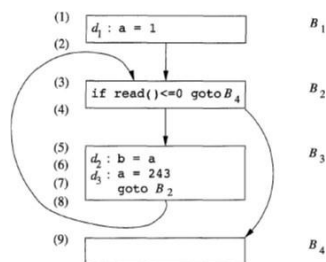
当我们分析程序的行为时，我们必须考虑程序执行时可能采取的各种通过程序的流图的程序点序列（“路径”）。然后，我们从各个程序点上可能的程序状态中提取需要的信息，解决特定的数据流分析问题。在更复杂的分析中，我们必须考虑在调用和返回执行时会形成在各种不同过程的流图之间跳转的路径。但是，要刚开始我们的分析研究时，我们将关注穿越单个过程的单个流图的路径。

让我们看看流图会告诉哪些关于可能执行路径的信息。

- 在一个基本块内，一条语句之后的程序点是与它的下一条语句之前的程序点相同
- 如果从块 B_1 到块 B_2 之间有一条边，则 B_2 的第一个语句之前的程序点可能紧跟在 B_1 的最后一条语句的程序点之后。

The Data-Flow Abstraction

- An **execution path** from point p_1 to point p_n is a sequence of points $p_1, p_2, \dots, p_n$ such that for each $i = 1, 2, \dots, n-1$, **either**
 - p_i is the point immediately preceding a statement and p_{i+1} is the point immediately follow that statement, **or**
 - p_i is the end of a block and p_{i+1} is the beginning of a successor block



The shortest complete execution path: (1, 2, 3, 4, 9)

The next shortest path executing one iteration of the loop: (1, 2, 3, 4, 5, 6, 7, 8, 3, 4, 9)

因此，我们可以把从点 p_1 到点 p_n 的一个执行路径（简称路径）定义为满足下列条件的点的序列点 $p_1; p_2; \dots; p_n$ ：对于每个 $i = 1; 2; \dots; n-1$,

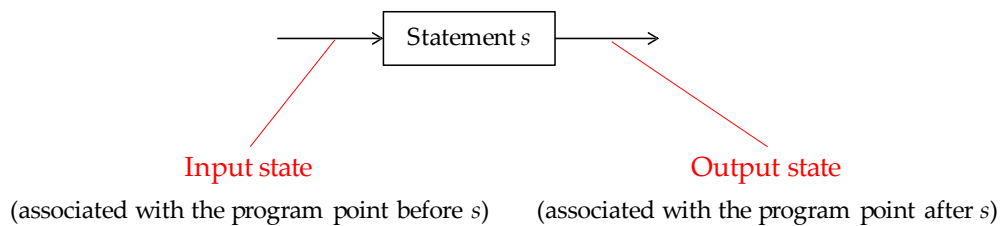
- 要么 p_i 是紧接在一条语句之前的点，并且 p_{i+1} 是紧接在该语句之后的点，
- 要么 p_i 是某个块的末尾，并且 p_{i+1} 是该基本块的一个后继块的开始。

一般来说，程序中可能存在无穷多条可能的执行路径，并且执行路径的长度并没有上限。程序分析把可能出现在某个程序点的所有程序状态总结为有穷的特性集合。不同的分析可以选择抽象掉不同的信息，并且一般来说，没有哪个分析会给出状态的完全表示。

例 9.8：即使是图 9.12 中的简单程序也描述了无限多个执行路径。最短的完整执行路径由程序点 (1; 2; 3; 4; 9) 组成，它不进入任何循环。次短的路径执行一次循环，它由程序点 (1; 2; 3; 4; 5; 6; 7; 8; 3; 4; 9) 组成。在这个例子中，我们知道在第一次执行程序点(5)时，由于 d_1 的定值， a 的值必然为 1。我们说 d_1 在第一次迭代的时候到达了点 (5)。在后续迭代中， d_3 到达了点 (5)， a 的值为 243。

The Data-Flow Abstraction

- **Program execution** can be viewed as *a series of transformations of the program state* (the values of all variables)
 - Each execution of an intermediate-code statement transforms an **input state** to a new **output state**



程序执行可视为程序状态（所有变量的值）的一系列转换

中间代码语句的每次执行都会将输入状态转换为新的输出状态

The Data-Flow Abstraction

- When analyzing a program, we must consider all the possible execution paths through a flow graph
 - In general, there is **an infinite number** of possible paths due to the existence of loops (and recursions)
- Data-flow analyses **summarize all the possible program states** that can occur at a program point with **a finite set of facts**
 - Different analyses may choose to abstract out different information
 - In general, no analysis is necessarily a perfect representation of the state

分析程序时，我们必须考虑流图中所有可能的执行路径

一般来说，由于循环（和递归）的存在，可能存在无限多的路径

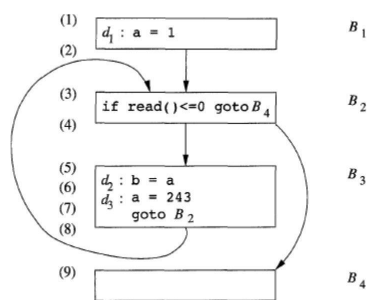
数据流分析用一组有限的事实概括了程序点上可能出现的所有程序状态

不同的分析可能会选择抽取不同的信息

一般来说，任何分析都不一定能完美地反映现状

一般来说，跟踪所有可能路径的所有程序状态是不可能。在数据流分析中，我们不区分到达一个程序点的路径之间的差异。此外，我们并不跟踪整个状态；相反，我们抽象出某些细节，只保留分析所需的数据。两个例子将说明一个程序点上的同一个状态可以导出不同的抽象信息。

Example (1)



Reaching definitions:

- The first time point (5) is executed, the value of a is 1 (i.e., definition d_1 reaches (5) in the first iteration)
- In subsequent iterations, d_3 reaches point (5) and the value of a is 243
- So, we may summarize all the program states at (5) by saying that the value of a is one of $\{1, 243\}$ and defined by one of $\{d_1, d_3\}$

到达定值：可能沿着某些路径到达某个程序点的定值称为到达定值。

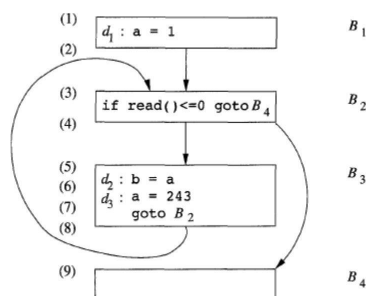
为了帮助用户调试程序，我们可能希望找出变量在某个程序点上可能具有哪些值，以及这些值可能在哪里定值。例如，

在执行第一个时间点 (5) 时， a 的值为 1（即定值 d_1 在第一次迭代中达到 (5)

在随后的迭代中， d_3 到达点 (5)， a 值为 243

因此，我们可以对在程序点（5）上的所有程序状态进行如下总结：a 的值总是{1, 243}中的一个，并且它由 {d1;d3}中的一个定值。

Example (2)



Constant folding:

- For constant folding, we wish to find those definitions that are the unique definition of their variable to reach a given program point, regardless of the execution path
- For this task, we may simply describe some variables as “constant” or “not a constant” instead of collecting their possible values
 - a is not a constant at point (5) and thus cannot be replaced by a constant

常量折叠。

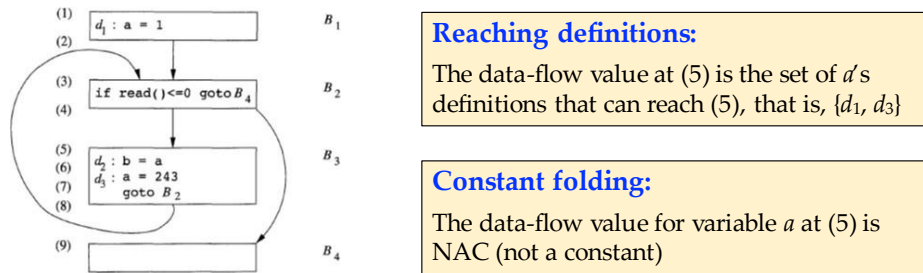
• 如果对变量 x 的某次使用只有一个定值可以到达，并且该定值把一个常量赋值给 x，那么我们可以简单地把 x 替换为该常量。另一方面，如果有多个对 x 的定值可以到达某个程序点，那么我们就无法对 x 执行常量折叠转换。因此，为了进行常量折叠，我们希望找到这样的定值：对于某个给定的程序点，不管执行哪条路径，它们都是唯一到达该点的对应变量的定值。

• 对于图 9.12 的点 (5)，没有哪个定值是到达该点的对 a 的唯一一定值，因此对于点 (5) 上的 a 来说，该集合是空的。即使一个变量在某个点上被唯一定值，该定值必须把为一个常量赋值给该变量，才可能进行常量折叠。这样，我们可以简单地把某些变量描述为“非常量 NAC”，而不是记录它们所有可能的取值或所有可能的定值。

因此，我们看到，根据分析目的，相同的信息可以通过不同的方式进行概括。

The Data-Flow Analysis Schema

- We associate with every program point a *data-flow value* that represents an abstraction of the set of all possible program states observed for that point*



* In each application of data-flow analysis, the set of possible data-flow values is the domain for the application.

在所有数据流分析应用中，我们都会把每个程序点与一个数据流值相关联，这个值是在该点可能观察到的所有程序状态的集合的抽象表示。所有可能的数据流值集合称为这个数据流应用的域。

例如，用于达到定值的数据流值的域是程序的定值集合的所有子集的集合。某个数据流值是一个定值的集合，我们希望将程序中的每个点与可能到达该点的定值的精确集合关联起来。

达到定值：

常量折叠：

如上所述，对于抽象方式的选择取决于分析的目标。为了提高效率，我们只跟踪相关的信息。

The Data-Flow Analysis Schema

- $IN[s]$: The data-flow value **before** the statement s
- $OUT[s]$: The data-flow value **after** the statement s
- The *data-flow problem* is to find a solution to a set of constraints on the $IN[s]$'s and $OUT[s]$'s for all statements s
 1. Constraints based on the semantics of the statements ("*transfer functions*")
 2. Constraints based on the flow of control

$IN[s]$: 语句 s 之前的数据流值

$OUT[s]$: 语句 s 之后的数据流值

数据流问题是为所有语句 s 的 $IN[s]$'s 和 $OUT[s]$'s 找到一组约束条件的解决方案。

基于语句语义的限制（“传递函数”）

基于控制流的限制

Transfer Functions (传递函数)

- The data-flow values before and after a statement are constrained by the semantics of the statement
- **Example:** consider the constant folding case (analyzing whether a variable is a constant at a program point)
 - Statement s : $x = 3$
 - Three variables: x, y, z
 - If $\text{IN}[s]$ is $x: \text{NAC}; y: 7; z: 3$, then $\text{OUT}[s]$ is $x: 3; y: 7; z: 3$
 - What if the statement is $x = y + z$ or $x = x + y$?

语句前后的数据流值受语句语义的限制。例如，假设我们的数据流分析涉及确定各个程序点处各变量的常量值。如果变量 a 在执行语句 $b = a$ 之前的值为 v ，那么在该语句之后 a 和 b 的值都为 v 。一个赋值语句之前和之后的数据流值的关系被称为传递函数。

示例：考虑常量折叠情况（分析变量在程序点是否为常量）

语句 s : $x = 3$

三个变量: x, y, z

如果 $\text{IN}[s]$ 是 $x: \text{NAC}; y: 7; z: 3$ ，则 $\text{OUT}[s]$ 为 $x: 3; y: 7; z: 3$

如果语句是 $x = y + z$ 或 $x = x + y$ 呢？

Transfer Functions (传递函数)

- The relationship between the data-flow values before and after each statement is known as a *transfer function*
- In a **forward-flow problem** (information propagate forward along execution paths), the transfer function f_s takes the data-flow value before the statement s and produces a new data-flow value after s
 - $OUT[s] = f_s(IN[s])$
- In a **backward-flow problem** (information flow backwards up the execution paths), the transfer function f_s takes the data-flow value after the statement s and produces a new data-flow value before s
 - $IN[s] = f_s(OUT[s])$

每条语句前后的数据流值之间的关系称为传递函数

在前向数据流问题（信息沿执行路径向前传播）中，一个语句 s 的传递函数 f_s 以语句 s 之前的数据流值作为输入，并产生语句 s 之后的新的数据流值。

$$OUT[s] = f_s(IN[s])$$

在逆向数据流问题（信息流沿着执行路径逆向流动）中，一个语句 s 的传递函数 f_s 把一个语句 s 之后的数据流值转变成为语句之前的新的数据流值。

$$IN[s] = f_s(OUT[s])$$

Control-Flow Constraints

1. **Within a basic block** of statements $s_1, s_2, \dots, s_n$, the data-flow value out of s_i is the same as that into s_{i+1}
 - $IN[s_{i+1}] = OUT[s_i]$, for all $i = 1, 2, \dots, n-1$
2. **Control-flow edges between basic blocks may create more complex constraints**
 - For example, in reaching definitions analysis, the set of definitions reaching the leader statement of a block should be the union of the definitions after the last statements of each of the predecessor blocks

关于数据流值的约束是从控制流中得到的。

在基本块内控制流很简单。如果块 B 按该顺序由语句 $s_1; s_2; \dots; s_n$ 组成，则 s_i 输出的控制流值和输入 s_{i+1} 的控制流值相同。即， $IN[s_{i+1}] = OUT[s_i]$ ，对于所有 $i = 1; 2; \dots; n$

1.

然而，基本块之间的控制流边会生成一个基本块的最后一个语句和后继基本块的第一个语句之间的约束，这些约束更加复杂。例如，如果对可能达到一个程序点的所有定值感兴趣，那么到达基本块的首语句的定值的集合就是到达它的各个前驱基本块的最后一个语句之后的定值的集合。下一节详细介绍数据如何在块之间流动。

Data-Flow Schemas on Basic Blocks (The Intra-Block Case)

- Within a basic block, control flows from the beginning to the end without interruption or branching
 - This allows us to restate the data-flow schema in terms of data-flow values entering and leaving the blocks (instead of before/after each statement)
- For a block B consisting of statements $s_1, s_2, \dots, s_n$, we have*
 - $IN[B] = IN[s_1]$
 - $OUT[B] = OUT[s_n]$
 - $OUT[B] = f_B(IN[B])$, where $f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$ (composing transfer functions)

* For backward-flow problem, $IN[B] = f_B(OUT[B])$, where $f_B = f_{s_1} \circ f_{s_2} \circ \dots \circ f_{s_n}$

从技术上讲，数据流模式涉及程序中每个点的数据流值，但我们如果认识到块内的数据流处理通常很简单，就可以节约数据流分析所需的时间和空间。在一个基本程序块内，控制从开始流向结束，没有中断或分支。

这样，我们就可以根据数据流值进出数据块（而不是每条语句之前/之后）来重述数据流模式。

假设块 B 按顺序由语句 $s_1 ; \dots ; s_n$ 组成。

如果 s_1 是基本块 B 的第一条语句，则 $IN[B] = IN[s_1]$ ，

同理，如果 s_n 是基本块 B 的最后一条语句，则 $OUT[B] = OUT[s_n]$ 。

基本块 B 的传递函数（用 f_B 表示）可以通过组合块中语句的传递函数来导出。也就是说，令 f_{s_i} 为语句 s_i 的传递函数，那么 $f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$ 。块的开始和结束之间的数据流值关系为 $OUT[B] = f_B(IN[B])$ ：

对于由语句 $s_1, s_2, \dots, s_n$ 组成的基本块 B ，我们有*

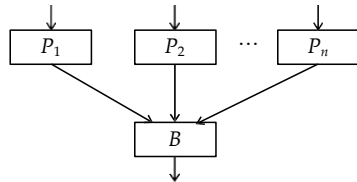
$$IN[B] = IN[s_1]$$

$$OUT[B] = OUT[s_n]$$

$$OUT[B] = f_B(IN[B]), \text{ 其中 } f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1} \text{ (组合传递函数)}$$

* 对于后向流动问题, $IN[B] = f_B (OUT[B])$, 其中 $f_B = f_{s_1} \circ f_{s_2} \circ \dots \circ f_{s_n}$

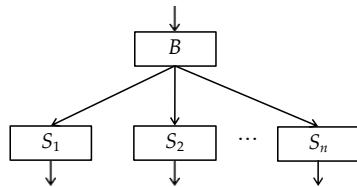
Data-Flow Schemas on Basic Blocks (The Inter-Block Case)



Forward-flow problem:

$$IN[B] = U \qquad OUT[P]$$

P a predecessor of B



Backward-flow problem:

$$OUT[B] = U \qquad IN[S]$$

S a successor of B

The operator U depends on specific problems.

因基本块之间的控制流而产生的约束可以很容易地通过重写得到, 把原来约束中的 $IN[B]$ 和 $OUT[B]$ 用 $IN[s_1]$ 和 $OUT[s_n]$ 代替即可。例如, 如果一个数据流值表示的是可能被赋予某个变量的常量集合, 那么我们就得到一个前向流问题, 其中

$$IN[B] = U \qquad OUT[P]$$

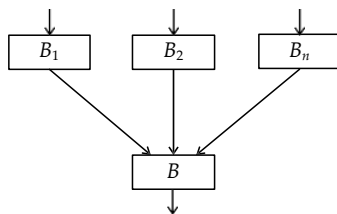
P a predecessor of B

我们将在活跃变量分析时看到逆向数据流问题, 逆向数据流问题的方程是相似的, 但 IN 和 OUT 的角色被调换了。即

$$OUT[B] = U \qquad IN[S]$$

S a successor of B

Example (Constant Folding Problem)



If:

$\text{OUT}[B_1]: x: 3; y: 4; z: \text{NAC}$

$\text{OUT}[B_2]: x: 3; y: 5; z: 7$

$\text{OUT}[B_3]: x: 3; y: 4; z: 7$

then:

$\text{IN}[B]: x: 3; y: \text{NAC}; z: \text{NAC}$

常量折叠问题 NAC 非常量 not a constant

Solutions to Data-Flow Equations

- Data-flow equations usually do not have a unique solution
 - All data-flow schemas compute **approximations** to the ground truth
- Our goal is to find the most “precise” solution that satisfies both **control-flow** and **transfer** constraints
 - Being precise enables valid code improvements (in general, a higher precision leads to a better improvement)
 - Satisfying constraints guarantees the safety of transformations (does not change program semantics)

与线性算术方程不同，数据流方程通常没有唯一解。所有数据流模式都计算基本事实的近似值。

我们的目标是找到一个最 “精确的 ” 的解决方案，同时满足控制流和传输约束条件。也就是说，我们需要一个解，它能够支持有效的代码改进，但是又不会导致不安全的转换。这些不安全的转换改变了程序计算的内容。

精确的代码可实现有效的代码改进（一般来说，精确度越高，改进效果越好）

满足约束条件可保证转换的安全性（不改变程序语义）

Reaching Definitions

- A definition d of some variable x *reaches* a point p if **there is a path** from the point immediately following d to p , such that **d is not “killed”** along that path
 - d is *killed* if there is any other definition of x along the path
 - Intuitively, if d reaches the point p , then d might be the place at which the value of x used at p was **last defined**

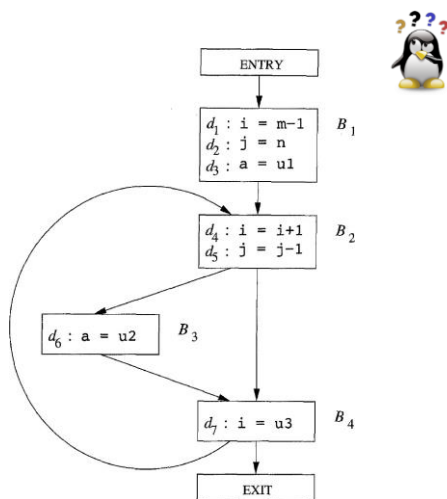
“到达定值”是最常见和最有用的数据流模式之一。通过了解当控制到达每个点时，每个变量 x 可能在程序中的何处定值，我们可以确定许多有关 x 的性质。

如果有一条从紧随定值 d 之后的程序点到达某一程序点 p 的路径，并且在这条路径上 d 没有被 “杀死”，我们说定值 d 到达程序点 p 。如果在这条路径上有对变量 x 的其他定值，我们就说变量 x 的这个定值被杀死了。

直观地说，如果某个变量 x 的一个定值 d 到达点 p ，则 p 处使用的 x 的值可能就是由 d 最后定值的。

变量 x 的一个定值（可能）将一个值赋给 x 的语句。过程参数、数组访问和间接引用都可能另有别名，因此指出一个语句是否指向特定程序变量 x 赋值并不是件容易的事情。程序分析必须保守；如果我们不知道一个语句 s 是否给 x 赋了一个值，我们必须假设它可能对 x 赋值；也就是说，语句 s 之后，变量 x 的值可能还是 s 执行之前的原值，但也可能变成了 s 所产生的新值。为了简单起见，本章的其余部分假设我们只处理没有别名的变量。此类变量包括大多数语言中的所有局部标量变量；对于 C 和 C++，有些局部变量的地址会被计算出来，这种局部变量不属于这类变量。

Example



Which definitions reach the block B_2 ?

- d_1, d_2, d_3 reach B_2
- d_5 reaches B_2 since there is no other definition to j in the loop
- d_4 does not reach B_2 since i is always redefined by d_7 (i.e., d_7 reaches B_2)
- d_6 reaches B_2

图 9.13 所示是一个有 7 个定值的流图。

让我们关注所有到达块 B_2 的定值。所有在块 B_1 中的定值都到达了块 B_2 的开头。

因为在转回基本块 B_2 的循环中找不到其他对 j 的定值，块 B_2 中的定值 $d_5: j = j-1$ 也可以到达基本块 B_2 的开头。但，这个定值杀死了定值 $d_2: j = n$ ，使得 d_2 不能到达 B_3 或 B_4 。

B_2 中的语句 $d_4: i = i+1$ 并未到达 B_2 的开头，这是因为变量 i 总是被 $d_7: i = u3$ 重新定值。

最后，定值 $d_6: a = u2$ 也能够到达块 B_2 的开头

Reaching Definitions

- For reaching definitions analysis, we **allow inaccuracies**. However, the decisions should be *"safe"*.
 - It is ok that some inferred reaching defs cannot actually reach a point
 - However, those defs that can reach a point should be identified
 - In other words, over-approximations are acceptable
- ***Reason of inaccuracies:*** In general, to decide whether each path in a flow graph can be taken is an undecidable problem
 - We often simply assume that every path in the flow graph can be followed in some execution of the program

我们在前面定义的到达定值，有时会允许一定的不精确。但是，它们都是在“安全”或“保守”的方向上不精确。例如，请注意我们假设一个流图的所有边都可以通过。在实践中这一假设可能并不成立。

一般来说，决定流图中的每条路径是否可以被执行是一个不可判定的问题。因此，我们简单地假设流图中的每条路径都可能在程序的某些执行时通过。在大部分到达定值的应用中，在一个定值不可能到达某个点的情况下，假设其能够到达是保守的。因此，我们可以允许在程序实际执行中根本不会被遍历的路径，我们也可以安全地允许定值穿越某个对同一变量的不明确定值。

Transfer Equations

- Consider a statement $d: u = v + w$
 - It *generates* a definition d of variable u and *kills* all other definitions of u
- Transfer function of a statement is in *gen-kill form*:
 - $f_d(x) = gen_d \cup (x - kill_d)$
 - x is the data-flow value (reaching definitions) before the statement
 - gen_d is the set of generated definitions, i.e., $\{d\}$
 - $kill_d$ is the set of killed definitions, i.e., all other definitions of u

达到定值的传递方程 我们现在将为达到定值问题设置约束。我们首先检查单个语句的细节。考虑一个定值 $d: u = v + w$ 在这里，以及在下面的内容中， $+$ 经常代表了一个一般性的二元运算符。以后我们经常这样做。

该语句“生成”一个变量 u 的定值并“杀死”程序中其他对 u 的定值，而进入这个语句的其他定值不受影响。因此，定值 d 的传递函数可以表示为 $f_d(x) = gen_d \cup (x - kill_d)$

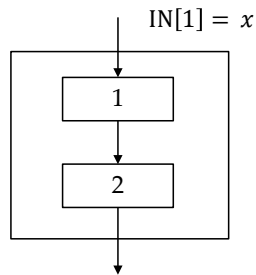
x 是该语句前的数据流值

$gen_d = \{d\}$ ，即由该语句生成的定值集合，

$kill_d$ 是程序中所有其他对 u 的定值

Transfer Equations

- Transfer function of a basic block is also in *gen-kill form*
- Consider a block of only two statements:



$$OUT[1] = f_1(x) = gen_1 \cup (x - kill_1)$$

$$OUT[2] = f_2(IN[2]) = f_2(OUT[1])$$

$$= gen_2 \cup (OUT[1] - kill_2)$$

$$= gen_2 \cup ((gen_1 \cup (x - kill_1)) - kill_2)$$

$$= gen_2 \cup (gen_1 - kill_2) \cup (x - (kill_1 \cup kill_2))$$

基本块的传递函数可以通过把它包含的所有语句的传递函数组合起来而构造得到。

(9.1) 形式的函数组合，我们将其称为“生成-杀死”形式，

Transfer Equations

- Generalize to a block B with n statements, we have:

$$f_B(x) = gen_B \cup (x - kill_B)$$

where

$$kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n$$

and

$$\begin{aligned}
 gen_B = & gen_n \cup (gen_{n-1} - kill_n) \cup \\
 & (gen_{n-2} - kill_{n-1} - kill_n) \cup \\
 & \dots \cup (gen_1 - kill_2 - kill_3 - \dots - kill_n)
 \end{aligned}$$

kill_B

The *kill set* is the union of all the defs killed by the individual statements

gen_B

The *gen set* contains all the defs inside the block that are *downward exposed* (visible immediately after the block)

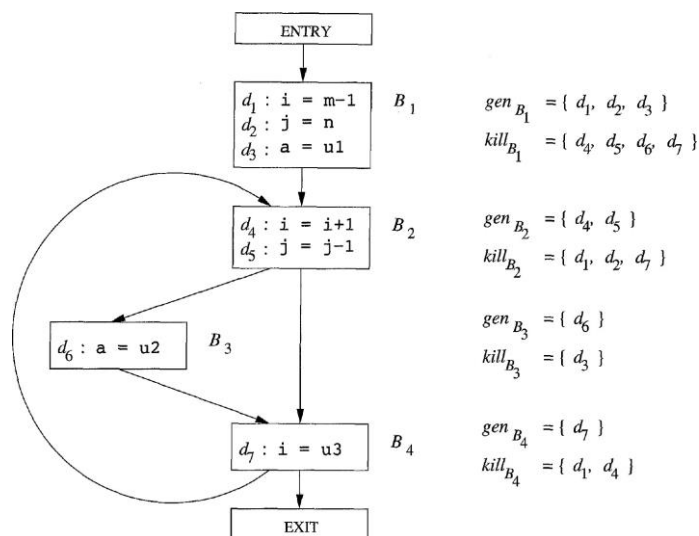
该规则可以扩展到由任意多个语句组成的块。假设块 B 有 n 个语句，而第 i 个语句的传递函数为 $f_i(x) = \text{gen}_i \cup (x - \text{Kill}_i)$ for $i = 1; 2; \dots; n$ 。那么块 B 的传递函数可以写为：

因此，和单个语句一样，一个基本块也会生成一个定值集并杀死一个定值集合。

集合 gen 中包含了所有在紧靠块之后的点上“可见”的该基本块中的定值——我们将它们称为向下可见。在一个基本块中，一个定值是向下可见的，仅当它没有被同一个基本块中较后的对同一变量的定值“杀死”。

一个基本块的 kill 集就是所有被块中各个语句杀死的定值的集合。请注意，一个定值可能会同时出现在基本块的 gen 集和 kill 集中。在这种情况下，该定值会被这个基本块生成，即优先考虑该定值是否在 gen 集中。这是因为在 gen-kill 形式中， kill 集在 gen 集之前被应用。

Example (1)



Example (2)

- Consider the basic block:

$d_1: a = 3$
$d_2: a = 4$

- The *gen* set: $\{d_2\}$
- The *kill* set contains d_1, d_2 and all other definitions to a

d_2 appears in both the *gen* and *kill* set of the block. Nonetheless, since the subtraction of the kill set precedes the union operation, the result of the transfer function for this block always includes d_2 .

基本块 $d_1: a = 3$ $d_2: a = 4$ 的 *gen* 集是 $\{d_2\}$ ，因为 d_1 不是向下可见的。

基本块 *kill* 集包含了 d_1 和 d_2 ，因为 d_1 杀死了 d_2 ， d_2 杀死了 d_1 。尽管如此，由于减去 *kill* 集的运算先于和 *gen* 集的并集运算，因此该块的传递函数的结果中总是包含定值 d_2

Control-Flow Equations

$$\text{IN}[B] = \bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$$

The *meet operator*

- **Rationale:**

- A definition reaches a program point as long as there exists at least one path along which the definition reaches. This explains why $\text{OUT}[P] \subseteq \text{IN}[B]$.
- Since a definition cannot reach a point unless there is a path along which it reaches, $\text{IN}[B]$ needs to be no larger than the union of the reaching definitions of all the predecessor blocks

控制流方程 接下来，我们考虑从基本块之间的控制流得到的约束的集合。因为只有一个定值能够沿着至少一条路径到达某个程序点，那么这个定值就到达该程序点。

- 因此只要存在一条从 P 到 B 的控制边，就会出现 $\text{OUT}[P] \subseteq \text{IN}[B]$ 。
- 然而，一个定值到达某个程序点的必要条件是它能够沿着某条路径到达这个程序点，因此 $\text{IN}[B]$ 不应该大于 B 的所有前驱基本块出口点的到达定值的并集。也就是说，可以安全地假设如下的方程式成立

我们将并集运算称为达到定值的交汇 *meet* 运算。在任何数据流模式中，我们使用交汇 *meet* 运算符来汇总各条路径会合点上不同路径所作的贡献。

Problem Formulation

- The reaching definitions problem is defined by the following equations:
 - For the ENTRY block: $OUT[ENTRY] = \emptyset$
 - Since no definitions reach the beginning of a program
 - For all basic blocks B other than ENTRY:
 - $OUT[B] = gen_B \cup (IN[B] - kill_B)$
 - $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P]$

我们假设每个控制流图都有两个空的基本块，一个入口节点（代表图的起始点）和一个出口节点（所有离开图的控制流都流向它）。由于没有定值到达图的开头，因此基本块 ENTRY 的传递函数是一个简单的常函数，它返回空集，即 $OUT[entry] = \emptyset$ 。

到达定值问题使用下面的方程定义：

The Iterative Algorithm

- **Input:** A flow graph with $kill_B$ and gen_B computed for each block B
- **Output:** $IN[B]$ and $OUT[B]$ for each block B

```
1)  OUT[ENTRY] =  $\emptyset$ ;  
2)  for (each basic block  $B$  other than ENTRY) OUT[ $B$ ] =  $\emptyset$ ;  
3)  while (changes to any OUT occur)  
4)      for (each basic block  $B$  other than ENTRY) {  
5)          IN[ $B$ ] =  $\bigcup_{P \text{ a predecessor of } B} OUT[P]$ ;  
6)          OUT[ $B$ ] =  $gen_B \cup (IN[B] - kill_B)$ ;  
      }
```

算法的结果是方程的最小不动点，即，对于各个 IN 和 OUT ，这个解给出的值总是此方程组的其他解所给出的值的子集。下面的算法的结果是可接受的，因为在某个集合 IN 或集合 OUT 中的定值确实可以到达该 IN 或 OUT 所描述的程序点。这个解也是我们所期望的，因为它不包含任何我们确定不会到达的定值。

算法 9.11：达到定值。

输入：一个流图，其中每个块 B 的 $kill_B$ 集和 gen_B 集都已计算出来。

输出：到达流图中各个基本块 B 的入口点和出口点的定值的集合，即 $IN[B]$ 和 $OUT[B]$

方法：我们使用迭代方法求解。一开始，我们估计对于所有 B 都有 $OUT[B] = \text{空}$ ，并逐步逼近想要的 IN 和 OUT 的值。因为我们必须不停迭代直到各个 IN 值（以及各个 OUT 值）收敛，我们可以使用一个布尔变量 $change$ 来记录每次扫描各基本块时，是否有 OUT 值发生更改。但是，在这个以及后面描述的类似算法中，我们假设用来跟踪更改情况的确切机制是可理解的，因此我们删除了这些细节。

该算法如图 9.14 所示。前两行初始化某些数据流值。第 (3) 行开始是一个循环，在循环中我们不停地迭代直至各个值收敛，第 (4) 行到第 (6) 行组成的内循环对入口结点之外的所有基本块应用数据流方程。

Algorithm Analysis

- *Why is the solution correct?*
 - The algorithm propagates definitions as far as they are not killed, thus simulating all possible executions of the program
- *Why will the algorithm eventually halt?*
 - For every B , $OUT[B]$ never shrinks; once a definition is added, it stays there forever
 - Since the set of all definitions is finite, eventually there will be an iteration during which nothing is added to any OUT , and the algorithm then terminates.

直观地说，为什么解决方案是正确的？

算法 9.11 会尽可能地传播定值，直到该定值被杀死，这样模拟了程序的所有可能的执行情况。

为什么算法最终会停止？

算法 9.11 最终会停止，因为对于每个 B ， $OUT[B]$ 决不会变小；一旦某个定值被加入到 OUT 值中，它就会一直待在那里。

由于所有定值的集合是有限的，因此最终必须有一次 `while` 循环的执行没有向任何 OUT 添加任何内容。此时算法终止了。在此时终止迭代是安全的，因为如果各个 OUT 值没有改变，下一趟中各 IN 值也不会改变。而如果各个 IN 值没有改变， OUT 值也不会改变，如此下去，所有后续的迭代都不会发生改变 IN 和 OUT 的值。

流图中的结点个数是 `while` 循环的迭代次数的上届。其理由是如果一个定值能够到达某个程序点，它必然可以通过无环的路径到达该点，而一个流图中的结点数量是无环路径中

节点数量的上限。在 while 循环的每次迭代中，每个定值至少沿着问题中的路径前进一个节点，并且根据结点在内层循环中被访问的顺序，它经常一次前进多个节点。

事实上，如果我们适当地安排第 (4) 行的 for 循环访问的块顺序，经验证据表明 while 循环的平均迭代次数低于 5（参见第 9.6.7 节）。由于定值集可以用位向量来表示，并且对这些集合的运算可以使用位向量上的逻辑运算来实现，所以算法 9.11 在实践中出奇地高效。

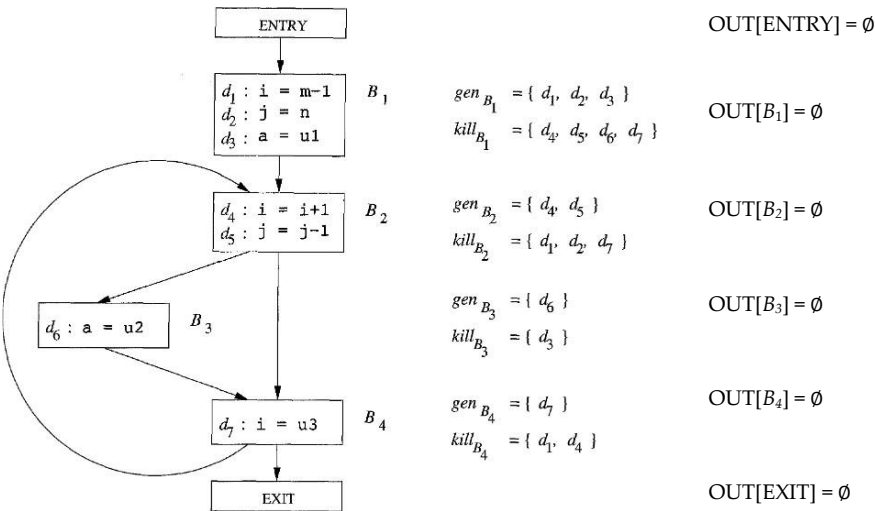
Example

```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4)   for (each basic block B other than ENTRY) {
5)     IN[B] =  $\bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$ ;
6)     OUT[B] =  $\text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$ ;
   }

```

Initialization:



我们将用位向量表示图 9.13 的流图中的七个定值 d₁;d₂;...;d₇，其中从左边开始的位 i 代表定值 d_i。集合的并集是通过相应位向量的逻辑 OR 来计算的。两个集合的差 S-T 通过首先对 T 的位向量求补，然后再将该补码与 S 的位向量进行逻辑 AND 来计算的。

图 9.15 的表中显示的是算法 9.11 中 IN 和 OUT 集取值。其初始值用上标 0 表示（如 OUT[B]₀ 中所示）它们由图 9.14 的第 (2) 行的循环赋值。它们都是空集，由位向量 000 0000 表示。

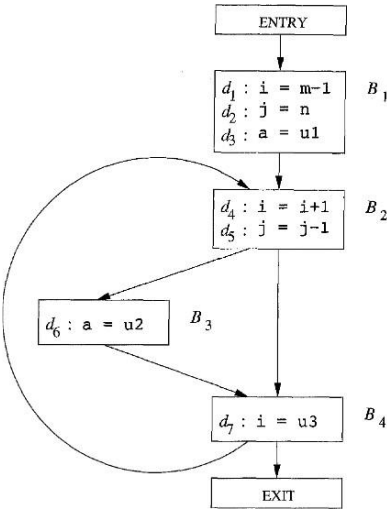
算法的后续迭代中的取值也使用上标表示，第一趟迭代的值标记为 IN[B]₁ 和 OUT[B]₁，第二个趟迭代的值标记为 IN[B]₂ 和 OUT[B]₂。

假设执行第 (4) 行至第 (6) 行的 for 循环，B 依次取值 B₁;B₂;B₃;B₄;EXIT。

当 $B = B_1$ 时, 由于 $OUT[entry] = \emptyset$, $IN[B_1]$ 是空集, 并且 $OUT[B_1]$ 是 gen_{B_1} 。该值与之前的值 $OUT[B_1]$ 不同, 因此我们知道在第一轮中有些值发生了变化 (因此会继续进行第二次循环)。

然后我们考虑 $B = B_2$ 并计算

Example



$gen_{B_1} = \{ d_1, d_2, d_3 \}$
 $kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$
 $gen_{B_2} = \{ d_4, d_5 \}$
 $kill_{B_2} = \{ d_1, d_2, d_7 \}$
 $gen_{B_3} = \{ d_6 \}$
 $kill_{B_3} = \{ d_3 \}$
 $gen_{B_4} = \{ d_7 \}$
 $kill_{B_4} = \{ d_1, d_4 \}$

```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4)   for (each basic block B other than ENTRY) {
5)     IN[B] = ∪P: a predecessor of B OUT[P];
6)     OUT[B] = genB ∪ (IN[B] - killB);
   }

```

Iteration #1:

$OUT[ENTRY] = \emptyset$

$IN[B_1] = \emptyset$

$OUT[B_1] = \{d_1, d_2, d_3\}$

$IN[B_2] = \{d_1, d_2, d_3\}$

$OUT[B_2] = \{d_3, d_4, d_5\}$

$IN[B_3] = \{d_3, d_4, d_5\}$

$OUT[B_3] = \{d_4, d_5, d_6\}$

$IN[B_4] = \{d_3, d_4, d_5, d_6\}$

$OUT[B_4] = \{d_3, d_5, d_6, d_7\}$

$IN[EXIT] = \{d_3, d_5, d_6, d_7\}$

$OUT[EXIT] = \{d_3, d_5, d_6, d_7\}$

在第一趟循环的最后, $OUT[B_2] = 001\ 1100$, 反映 B2 中生成了 d4 和 d5 的事实, 而 d3 到达了 B2 的开头并且没有在 B2 中被杀死。

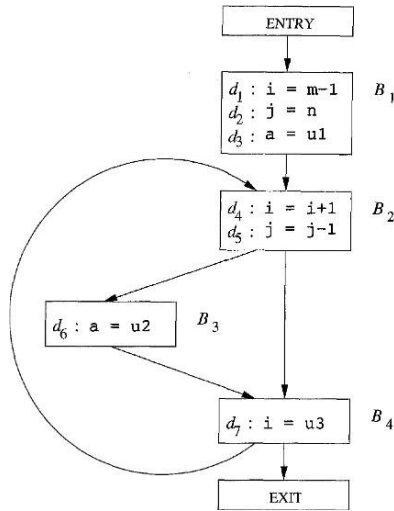
Example

```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4)   for (each basic block B other than ENTRY) {
5)     IN[B] =  $\bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$ ;
6)     OUT[B] = genB ∪ (IN[B] - killB);
   }

```

Iteration #2:



$gen_{B_1} = \{ d_1, d_2, d_3 \}$
 $kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$
 $gen_{B_2} = \{ d_4, d_5 \}$
 $kill_{B_2} = \{ d_1, d_2, d_7 \}$
 $gen_{B_3} = \{ d_6 \}$
 $kill_{B_3} = \{ d_3 \}$
 $gen_{B_4} = \{ d_7 \}$
 $kill_{B_4} = \{ d_1, d_4 \}$

OUT[ENTRY] = ∅

IN[B₁] = ∅

OUT[B₁] = {d₁, d₂, d₃}

IN[B₂] = {d₁, d₂, d₃, d₅, d₆, d₇}

OUT[B₂] = {d₃, d₄, d₅, d₆}

IN[B₃] = {d₃, d₄, d₅, d₆}

OUT[B₃] = {d₄, d₅, d₆}

IN[B₄] = {d₃, d₄, d₅, d₆}

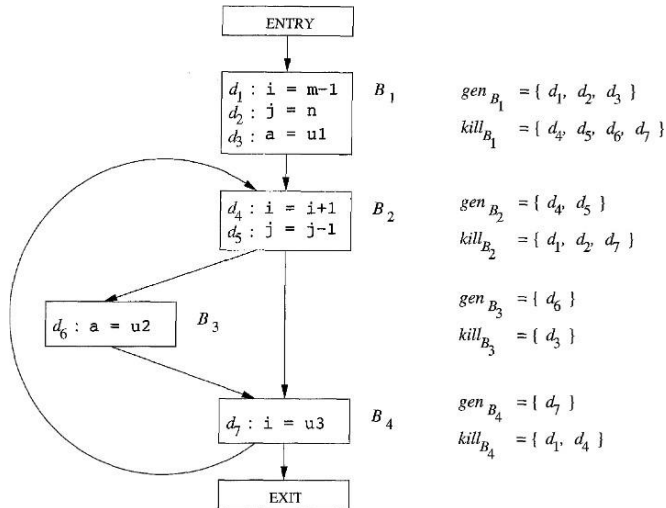
OUT[B₄] = {d₃, d₅, d₆, d₇}

IN[EXIT] = {d₃, d₅, d₆, d₇}

OUT[EXIT] = {d₃, d₅, d₆, d₇}

在第二轮之后，OUT[B₂] 已发生更改，反映了 d₆ 也到达 B₂ 的开头并且没有被 B₂ 杀死的事实。我们在第一趟中没有了解到这个事实，因为从 d₆ 到 B₂ 末尾的路径，即 B₃ → B₄ → B₂ 没有在一趟中被顺序经过。也就是说，当我们得知 d₆ 到达 B₄ 的末尾时，我们已经在第一趟中计算了 IN[B₂] 和 OUT[B₂]。

Example



```

1) OUT[ENTRY] = ∅;
2) for (each basic block B other than ENTRY) OUT[B] = ∅;
3) while (changes to any OUT occur)
4)   for (each basic block B other than ENTRY) {
5)     IN[B] = ∪P a predecessor of B OUT[P];
6)     OUT[B] = genB ∪ (IN[B] - killB);
  }

```

Iteration #3: fixed point!

OUT[ENTRY] = ∅

IN[B₁] = ∅

OUT[B₁] = {d₁, d₂, d₃}

IN[B₂] = {d₁, d₂, d₃, d₅, d₆, d₇}

OUT[B₂] = {d₃, d₄, d₅, d₆}

IN[B₃] = {d₃, d₄, d₅, d₆}

OUT[B₃] = {d₄, d₅, d₆}

IN[B₄] = {d₃, d₄, d₅, d₆}

OUT[B₄] = {d₃, d₅, d₆, d₇}

IN[EXIT] = {d₃, d₅, d₆, d₇}

OUT[EXIT] = {d₃, d₅, d₆, d₇}

第二趟之后，OUT 集中的所有值都没有变化。因此，算法在第三趟之后终止，此时各个 IN 和 OUT 的值 如图 9.15 的最后两列所示。

Live-Variable Analysis

- Aim to know for variable x and point p whether the value of x at p could be used on some path starting at p
 - If yes, we say x is *live* at p ; otherwise, x is *dead* at p .
- An important use of live-variable information is register allocation for basic blocks
 - It is not necessary to store a value computed in a register to memory at the end of a basic block
 - If all registers are used and we need another register, we should favor using a register with a dead value
- Another important use is to eliminate dead code
 - If a value is dead at a point, there is no need to compute it

- 有些代码改进转化所依赖的信息是按照程序中的控制流相反的方向计算的。

现在我们将研究一个这样的例子。在活跃变量分析中，我们希望知道对于变量 x 和程序点 p ， x 在点 p 上的值是否会在流图中的某条从点 p 出发的路径中是使用。如果是，我们就说 x 在 p 上活跃；否则，就说 x 在 p 处死亡。

- 活跃变量信息的一个重要用途是基本块的寄存器分配。第 8.6 节和第 8.8 节介绍了该问题的各个方面。

在一个值被计算并保存到一个寄存器中后，它很可能会在块中使用该值。如果它在基本块的结尾处是死的，就不必在结尾处保存这个值。

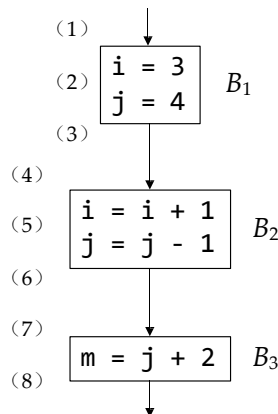
另外，如果所有寄存器都被占用时，如果我们还需要申请另一个寄存器，我们应该倾向于使用一个存放了已死亡的值的寄存器，因为这个值不需要保存到内存。

- 另一个重要的用途就是删除死代码

一个值在某个程序点是死的，就没必要计算它。

Example

Live: value could be used on subsequent path
Dead: value not referenced on subsequent path

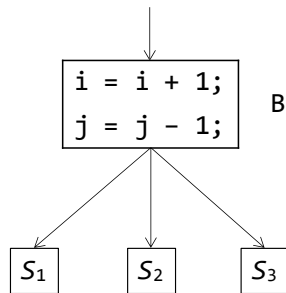


For the partial flow graph, we have:

Point	i	j	m
(1)	Dead	Dead	Dead
(2)	Live	Dead	Dead
(3)	Live	Live	Dead
(4)	Live	Live	Dead
(5)	Dead	Live	Dead
(6)	Dead	Live	Dead
(7)	Dead	Live	Dead
(8)	Dead	Dead	Dead

Obsevation: Live-variable analysis a **backward data-flow problem** since the subsequent defs/uses determine the liveness of a variable at a point

Data-Flow Equations



Data-flow values:

$IN[B]$: variables live at the entry of the block B

$OUT[B]$: variables live at the exit of block B

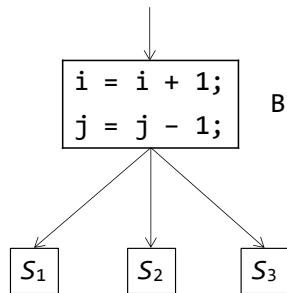
$$OUT[B] = \bigcup_{S \text{ is a successor of } B} IN[S]$$

Rationale: For safety, as long as a variable is live at the entry of any successor block of B , we consider the variable live at the exit of B

这里，我们直接以 $IN[B]$ 和 $OUT[B]$ 的方式定义数据流方程， $IN[B]$ 和 $OUT[B]$ 分别表示在紧靠块 B 之前和紧随 B 之后的点上的活跃变量集合。

为了安全起见，只要一个变量在 B 的任何后继块的入口处是活跃的，我们就认为该变量在 B 的出口处是活跃的。

Data-Flow Equations



$$IN[B] = use_B \cup (OUT[B] - def_B)$$

use_B: the set of variables whose values are used in *B* prior to any definition of the variable (*i*, *j*)

def_B: the set of variables defined in *B* prior to any use of that variable in *B* (i.e., the first time the variable appears, it appears as a definition; *def_B* is $\emptyset$ for the example)

这些方程可以通过以下的方法得到，首先定义各个语句的传递函数，然后再把它们组合起来得到一个基本块的传递函数。

我们给出下面的定义

1. *def B* 是指如下变量的集合，这些变量在 *B* 中的定值（即被明确地赋值）先于任何对它们的使用。
2. *useB* 是指如下变量的集合，它们的值可能在 *B* 中先于任何对它们的定值被使用。

Computing use_B and def_B

- **Transfer function for a statement:**
 - $s: x = y + z$
 - $use_s = \{y, z\}$
 - $def_s = \{x\} - \{y, z\}$ // y, z might be x
- **Transfer function for a block of statements $s_1, s_2, \dots, s_n$:**
 - $use_B = use_1 \cup (use_2 - def_1) \cup (use_3 - def_2 - def_1) \cup \dots \cup (use_n - def_1 - def_2 - \dots - def_{n-1})$
 - $def_B = def_1 \cup (def_1 - use_0) \cup (def_2 - use_0 - use_1) \cup \dots \cup (def_n - use_1 - use_2 - \dots - use_{n-1})$

Problem Formulation

- All variables are dead at the exit of a program
 - $IN[EXIT] = \emptyset$
- For all basic blocks B other than EXIT:
 - $IN[B] = use_B \cup (OUT[B] - def_B)$
 - $OUT[B] = \bigcup_{S \text{ is a successor of } B} IN[S]$
- Comparing to reaching definitions analysis:
 - Both sets of equations have union as the meet operator
 - Information flow for liveness travels “backward”

根据这些定义， $useB$ 中的任何变量都必然被认为在块 B 的入口处活跃，而 $def B$ 中变量在 B 的开头肯定是死的。实际上， $def B$ 中的成员 “杀死” 了某个变量可能因从 B 开始的某条路径而成为活跃变量的任何机会。

这样，将 def 和 use 与未知的 IN 和 OUT 值联系起来的方程定义如下： $IN[exit]=$ ；对于除 $exit$ 之外的所有基本块 B 来说， $IN[B]=useB[(OUT[B] \cap def B)]$ $OUT[B]=S[B] \cap IN[S]$

第一个方程描述了边界条件，即程序的出口处没有变量是活跃的。第二个方程说明一个变量要在进入一个基本块时活跃，必须满足下面两个条件中的一个：要么它在基本块中被重新定值之前就被使用，要么它在离开基本块时活跃且在基本块中没有对它重新定值。第三个方程表示，一个变量在离开一个基本块时活跃当且仅当它在进入该基本块的某个后继时活跃。

应该注意活跃性方程和到达定值方程之间的关系：

两组方程都以并集作为交汇运算。原因是，在各个数据流模式中，我们都沿着路径传播信息，并且我们只关心是否存在任何路径具有我们想要的性质，而不关心某些结论是否在所有的路径上都成立。

然而，活跃性信息流 “向后传播”，与控制流的方向相反。其中的原因时在这个问题中，我们想要确保在一个程序点 p 上对变量 x 的使用可以被传递到某个执行路径中 p 之前的所有程序点，这样我们才知道在前面的这些点上 x 的值会被使用。

The Iterative Algorithm

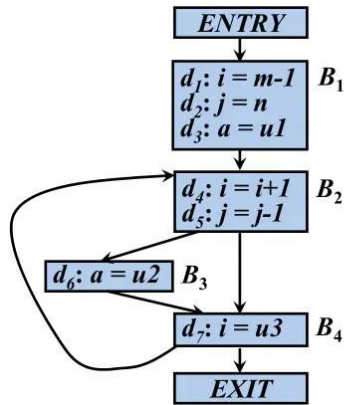
- **Input:** A flow graph with def_B and use_B computed for each block B
- **Output:** $IN[B]$ and $OUT[B]$ for each block B

```
IN[EXIT] =  $\emptyset$ ;  
for (each basic block  $B$  other than EXIT)  $IN[B] = \emptyset$ ;  
while (changes to any IN occur)  
    for (each basic block  $B$  other than EXIT) {  
         $OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$ ;  
         $IN[B] = use_B \cup (OUT[B] - def_B)$ ;  
    }
```

Similar to the iterative algorithm for reaching definitions analysis, $IN[B]$ never shrinks and its size is bounded, so the algorithm will eventually halt.

为了解决一个逆向传播的数据流问题，我们初始化 $IN[exit]$ 而不是 $OUT[entry]$ 。集合 IN 和 OUT 的角色互换，并且 use 和 def 分别替代 gen 和 $kill$ 。和到达定值问题一样，活跃性方程的解不必是唯一的，且我们希望得到具有最小活跃变量集合的解。解方程时使用的算法本质上是算法的逆向传播版本 9.11。

Example



$use_{B1} = \{ m, n, u1 \}$

$def_{B1} = \{ i, j, a \}$

$use_{B2} = \{ i, j \}$

$def_{B2} = \Phi$

$use_{B3} = \{ u2 \}$

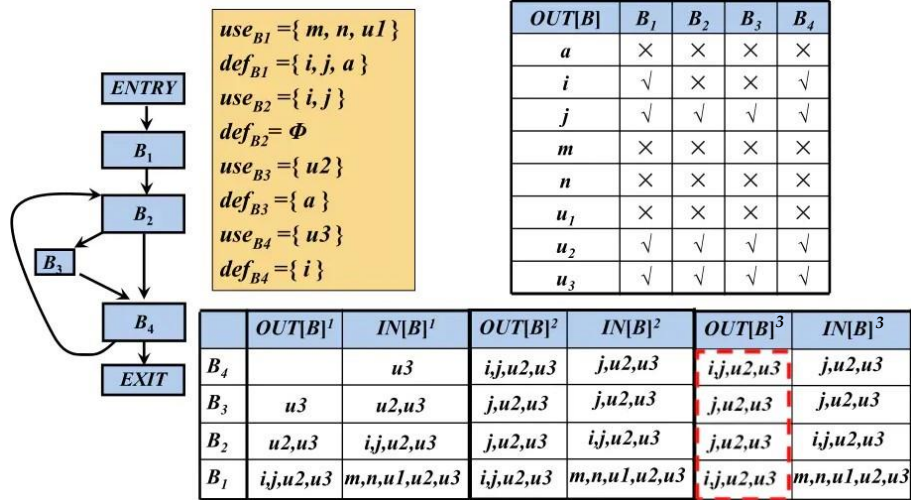
$def_{B3} = \{ a \}$

$use_{B4} = \{ u3 \}$

$def_{B4} = \{ i \}$

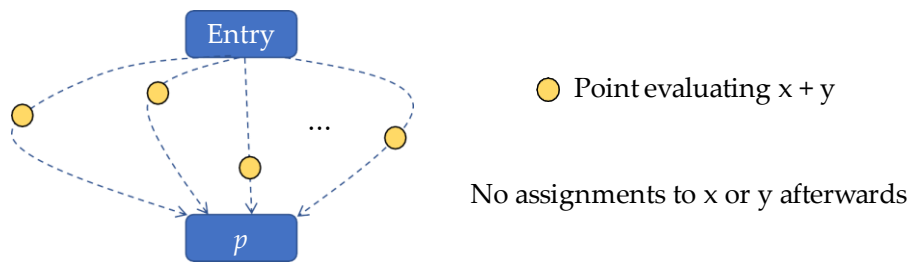
<https://www.jianshu.com/p/ebc1c72b881c>

Example



Available Expressions

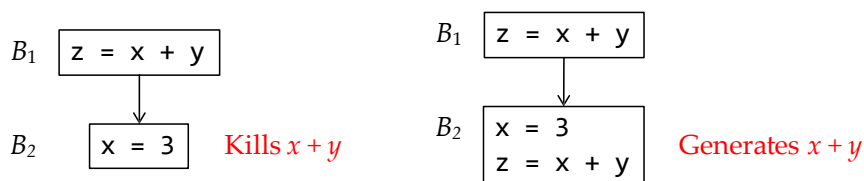
- An expression $x + y$ is available at a point p if **every path** from the entry node to p evaluates $x + y$, and after the last such evaluation prior to reaching p , there are **no subsequent assignments to x or y**



如果从入口节点 到达程序点 p 的每条路径都对表达式 $x + y$ 求值，且从最后一个这样的求值之后到 p 点的路径上没有再次对 x 和 y 赋值，那么 $x + y$ 在点 p 处可用。

Killing / Generating Expressions

- A block **kills** expression $x + y$ if:
 - It assigns x or y and does not subsequently recompute $x + y$
- A block **generates** expression $x + y$ if:
 - It evaluates $x + y$ and does not subsequently define x or y



对于可用表达式数据流模式，

我们说一个块对 x 或 y 赋值（或者可能对它们赋值）并且之后没有再重新计算 $x + y$ ，我们就说该基本块杀死了表达式 $x + y$ 。

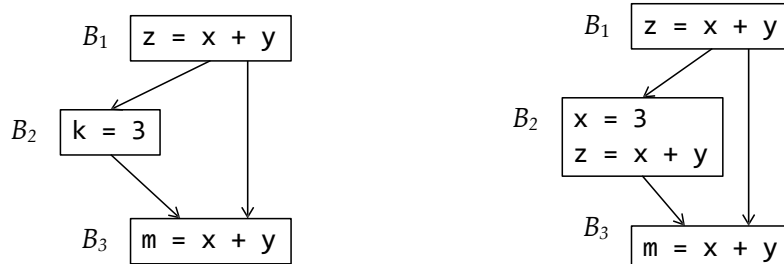
如果一个基本块确实对 $x + y$ 求值，并且之后没有再对 x 或 y 定值，那么这个基本块生成表达式 $x + y$ 。

请注意，“杀死”或“生成”一个可用表达式的概念与达到定值的概念并不完全相同。然而，这些“杀死”和“生成”的概念在行为上和到达定值中相应的概念在本质上是

一致的。

The Primary Use of Available Expressions

- Detecting global common expressions



Case 1: B_2 does not redefine x or y

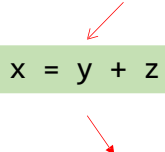
Case 2: B_2 redefines x and recomputes $x + y$

In both cases, $x + y$ needs not to be recomputed in B_3

可用表达式信息的主要用途是检测全局公共子表达式。

Computing Generated Expressions

S : the set of expressions available before the statement



$x = y + z$

Compute the expressions available after the statement:

- Add to S the expression of $y + z$
- Delete from S any expression involving variable x

If we repeat the steps for each statement in a block B , we will get the set of **generated expressions** for B after we process the last statement.

我们可以从头到尾地处理基本块内的各个语句，计算一个基本块内各个点上生成的表达式集合。在该块之前的点上没有生成任何表达式。如果在点 p 处可用有表达式的集合是 S ，且 q 是 p 之后的点，它们之间有语句 $x = y + z$ ，那么我们通过以下两个步骤可用得到点 q 上的可用的表达式集合。

1. 将表达式 $y + z$ 添加到 S 中。
2. 从 S 中删除任何涉及变量 x 的表达式。

请注意，因为 x 可能与 y 或 z 相同，所有上面这些步骤必须按正确的顺序完成。当我们到达块的末尾后， S 就是该块生成的表达式集合。

Computing Killed Expressions

- For a basic block B , the set of **killed expressions** is all expressions, say $y + z$, such that:
 - y or z is defined in B
 - $y + z$ is not generated by B

被杀死的表达式的集合是所有类似于 $y + z$ 的表达式，其中

- y 或 z 在块中被定值，
- 并且这个基本块没有生成 $y + z$ 。

Example

Statement	Available Expressions
	$\emptyset$
$a = b + c$	
	$\{b + c\}$
$b = a - d$	
	$\{a - d\}$
$c = b + c$	
	$\{a - d\}$
$d = a - d$	
	$\emptyset$

考虑图 9.18 的四个语句。

第一个语句之后 $b + c$ 可用。

第二条语句之后， $a - d$ 变得可用，但 $b + c$ 不再可用，因为 b 已被重新定值。

第三条语句不会使 $b + c$ 再次可用，因为 c 的值立即就被更改了。

最后一条语句之后， $a - d$ 不再可用，因为 d 已更改。

因此，这个基本块不会生成任何可用表达式，并且所有涉及 a 、 b 、 c 或 d 的表达式都会被杀死。

Data-Flow Equations

- At the exit of the entry node, there are expressions available:
 - $OUT[ENTRY] = \emptyset$
- For all basic blocks B other than ENTRY:
 - $OUT[B] = e_genB \cup (IN[B] - e_killB)$
 - $IN[B] = \bigcap_{P \text{ is a predecessor of } B} OUT[P]$

The equations are similar to those for reaching definition analysis except that the meet operator is intersection rather than union.*

* This is for safety reasons. Although producing a subset of the exact set of available expressions prevents certain optimizations, this does not lead to the change of program semantics

我们可以通过类似于计算到达定值的方式来找到可用的表达式。假设 U 是出现在程序中的一个或多个语句的右部的表达式的全集。对于每个块 B ，令 $IN[B]$ 表示在 B 的开始处可用的 U 中的表达式的集合。 $OUT[B]$ 表示在 B 的结尾处可用的表达式的集合。定义 e_genB 为 B 生成的表达式集合，而 e_KillB 为被 B 杀死的 U 中的表达式的集合。请注意， IN 、 OUT 、 e_gen 和 e_Kill 都可以用位向量表示。

以下方程给出了未知的 IN 和 OUT 值之间，以及它们和已知量 e_gen 和 e_Kill 之间的关系：

$OUT[entry] = \text{空};$

并且 对于除 $entry$ 之外的所有基本块，有 $OUT[B] = e_genB \cup (IN[B] - e_KillB)$

$IN[B] = \bigcap_{P \in \text{pred}(B)} OUT[P];$

上述方程看起来与达到定值的方程几乎相同。与到达定值一样，这个方程的边界条件也是 $OUT[entry] = \text{空};$ ，这是因为在 $ENTRY$ 的出口处没有任何可用的表达式。最重要的区别在于这个方程组的交汇运算是交集运算而不是并集运算。因为只有当一个表达式在一个基本块的所有前驱的结尾处都可用，它才会在该基本块的开头可用，因此使用交集运算是正

确的。相反，只要一个定值到达了一个基本块的任何一个前驱的结尾处，它就到达该块的开头，所以在到达定值方程中使用并集运算作为交汇运算。

The Iterative Algorithm

- **Input:** A flow graph with e_kill_B and e_gen_B computed for each block B
- **Output:** $IN[B]$ and $OUT[B]$, the expressions available at the entry and exit of each block B

```
OUT[ENTRY] =  $\emptyset$ ;
for (each basic block  $B$  other than ENTRY)  $OUT[B] = U$ ;
while (changes to any  $OUT$  occur)
    for (each basic block  $B$  other than ENTRY) {
         $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$ ;
         $OUT[B] = e\_gen_B \cup (IN[B] - e\_kill_B)$ ;
    }
```

U : the universal set of all expressions

使用“交”而不是“并”使得可用表达式方程的表现与达到定值的方程组的表现不同。虽然这两个集合都没有唯一的解，但到达定值方程组的解是符合“到达”的定义的最小集合。在求解到达定值方程的过程中，我们首先假设任何地方都没有定值到达，然后逐渐增大到达定值的集合，最终构建得到该解。在这个方法里，除非找到一条能把某个定值 d 传播到某个点 p 的实际路径，否则我们从来不假设定值 d 可以到达点 p 。相反，对于可用的表达式方程组，我们希望得到具有最大可用表达式集合的解。因此，我们首先给出较大的近似值，然后逐步消减。

首先，我们假设在除了入口块的末尾之外的所有地方，所有表达式（即集合 U ）都是可用的。只有当我们发现有一条路径使得某个表达式不可用时，我们才删除这个表达式。这种方法看起来不是那么显而易见，但是我们可以得到了一组真正可用的表达式集合。在处理可用表达式时，保守的做法是生成可用表达式的精确集合的子集。所以说子集是保守的，是因为我们将这个信息用于把一个可用表达式的计算替换为之前计算得到的

值。不知道一个表达式可用只会阻碍我们改进代码，而把一个不可用的表达式认为可用则可能导致我们更改程序的计算结果。

Summary of The Problems

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Boundary	$OUT[ENTRY] = \emptyset$	$IN[EXIT] = \emptyset$	$OUT[ENTRY] = \emptyset$
Meet (A)	$\cup$	$\cup$	$\cap$
Equations	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigvee_{P, pred(B)} OUT[P]$	$IN[B] = f_B(OUT[B])$ $OUT[B] = \bigvee_{S, succ(B)} IN[S]$	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigvee_{P, pred(B)} OUT[P]$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$
Application	Constant folding	Dead code elimination	Global common subexpression elimination